CIE

COPYRIGHT INFORMATION EXTRACTION FORM

TITLE: Agentic AI–Driven Intelligent RPA Task Orchestrator for Automated Document Processing

SCHOOL: Chitkara University, Rajpura, Punjab

Name: Rishabh Jain

Roll No/ Employee Code: 2210990725

All MEMBERS DETAILS INCLUDING SUPERVISOR

NAME	EMP CODE/ROLL	ADDRESS WITH EMAIL AND MOBILE NO.
Rishabh Jain	2210990725	House no. 74, Sector 17, Panchkula, Haryana (134109) Mobile number - 9478026310

ABSTRACT

Robotic Process Automation (RPA) technology represents an innovative solution to automate routine rule-based tasks and enhance operational efficiency in corporate settings. Nevertheless, conventional RPA systems cannot operate autonomously since they depend on fixed workflows, pre-set rules, and execution logic defined at the design stage. The problem arises due to limited capabilities of traditional automation solutions when working with documents received by email (e.g., invoices, reports, resumes, purchase orders).

In business practice, documents come in various formats, layouts, and structures. Such materials need to be analyzed and processed accordingly. In turn, the lack of capability to recognize the nature of the document and choose the optimal processing workflow becomes a major obstacle for automated systems. Traditional RPA relies entirely on manual intervention when classifying documents and choosing the processing method.

Such a system suffers from limited flexibility and inefficiency as it cannot cope with large numbers of documents without human participation. Thus, there is a need to develop a flexible and autonomous system capable of analyzing the document, determining its nature, and deciding upon further actions to ensure smooth operation.

1.1 Objective of the Project

The key purpose of this project is to construct an intelligent document processing system that combines agentic AI decision making layer and the usual RPA processes. The goal of this project is to create a process where the document processing will be done automatically and intelligently without the necessity of choosing the workflows manually and managing them.

More specifically, the following purposes will be achieved:

- Automation of the document import from the emails
- Extraction of relevant data from the unstructured documents with the help of OCR technology
- Creation of the intelligent agent that will analyze documents
- Finding document type and the most suitable processing process
- Automation of the workflows according to the AI decision-making process
- Transparency and logging

1.2 Proposed Solution / System Overview

The suggested solution proposes an Agentic AI-Driven RPA Orchestration Framework that builds upon conventional automation capabilities by integrating an intelligent decision-making component.

The framework is implemented via a series of processes:

1. Email Ingestion Module

This process constantly checks a specified email account using robotic process automation (RPA) platforms such as UiPath. The process can identify new emails that come with attachments according to certain criteria (such as the subject line, sender, or inclusion of attachments). The email is then fetched and attachments are extracted without any human effort.

2. Document Ingestion and Archival

Once extracted, attachments are saved in a well-structured folder following a standardized naming convention (for example, timestamp or unique identifier).

3. OCR-Based Text Extraction Module

Text extraction from such sources as PDF documents and image files is done via specialized tools like Tesseract OCR. Prior to the conversion process, documents are preprocessed for

better accuracy. After the extraction, the obtained text is subject to cleaning up for further usage.

4. Agentic Decision Engine (Python-Based Classifier)

Then, an AI agentic module that performs document classification based on keyword scores and contextual understanding of information becomes operational. Using the Python-based classifier makes the decision engine more flexible and capable.

5. Decision Interpretation and Orchestration

The output received by the document classifier is returned to the RPA workflow. It allows interpreting the decisions made by AI engines properly and orchestrating them within the workflow.

6. Dynamic Workflow Execution

Depending on the classification output, documents are moved to appropriate folders. The key advantage of this solution is its ability to route the document in real time. No coding of certain conditions is required as the engine automatically creates needed directories and works with new document types.

7. Logging and Traceability Layer

All operations conducted by the automated system, from document processing to performed actions based on decision interpretation, are logged with time stamps and relevant identifiers to ensure traceability.

1.3 System's Main Features

The system employs a number of advanced technologies that set it apart from traditional automatic systems:

• Automated Email Watching

This solution uses robotic process automation software, such as UiPath, to constantly monitor the designated email inbox for incoming messages with document attachments, thereby enabling automatic document acquisition.

• Dynamic Document Processing Workflow

A completely automated workflow is created, encompassing all aspects ranging from document collection, its saving, processing, categorization, and routing.

• OCR-Based Text Extraction

In case of unstructured data, such as scanned PDFs and images, OCR technologies, like Tesseract OCR, are used to extract visual information from the document and convert it into readable text.

• Agent-Based Document Classification

A smart agent written in Python uses keyword scoring and contextual reasoning to analyze the text extracted by OCR software and identify the type of documents.

• Dynamic Folder Creation and Document Routing

Depending on the results of classification, dynamic folder creation and routing take place. It eliminates the need for static rules and provides adaptivity and flexibility.

• Event-Driven Workflow

The system processes new documents automatically upon their receipt. Emails and other new documents trigger automatic execution of the workflow.

• **Module-Based Approach**

The system is developed according to the principles of modularity, meaning that each step of the process, from ingestion through classification to routing, performs its task independently.

• Advanced Logging Functionality

Each activity carried out by the system, from text extraction to classification and routing, is logged to ensure proper monitoring and easy debugging of the workflow.

1.4 Technology / Method Used

The implementation of this technology uses the following methods:

- **UiPath RPA platform**

This tool is used for business process automation, emails processing, OCR integration, and orchestration

- **Python**

For creating an agentic document classification model

- **OCR technology (UiPath Document OCR)**

Used for extracting information from PDF and scans

- **Keyword-based document classifier algorithm**

Uses the scoring function to classify document types by the usage of keywords

- **Filesystem-based dynamic routing**

Used to automatically sort documents into separate folders depending on the document classification result

- **JSON/ TXT inter-process communication protocol**

1.5 Expected Outcome and Benefits

It offers the following benefits that are tangible and quantifiable:

- **Less Human Involvement**

No need for manually classifying documents and selecting workflow

- **Greater Efficiency**

Speed in processing larger numbers of documents

- **Increased Scalability**

Can handle more than one type of document without creating new workflows

- **Adaptability**

Automatically adapts to various types of documents and content

- **Accuracy**

Eliminates mistakes caused by humans in processing documents

- **Accountability**

Provides detailed logs which trace all the steps involved

- **Enterprise-Level Feasibility**

Is applicable to other enterprise scenarios such as finance, HR, and business processes

The solution is an example of intelligent automation which adds value to traditional RPA.

1.6 Original Contribution Statement

This research offers an innovative way to implement agentic artificial intelligence within Robotic Process Automation, with the help of a dynamic orchestration framework, which has the capability to make decisions independently.

Specifically, the following are among the contributions made through this research work:

- The implementation of a new classification module based on agents within Robotic Process Automation
- The realization of dynamic workflow orchestration without the use of static branches
- The development of a highly scalable architecture for document processing
- The demonstration of intelligent automation using AI with unstructured data

The work presented in this report is an important step towards developing intelligent automation.

1.7 IMPLEMENTATION METHODOLOGY

1. Email-Based Document Ingestion

The system is intended to run in an event-driven mode, where the events are detected in the form of document request emails in the inbox of a particular mailbox. The UiPath workflow uses the Get Outlook Mail Messages task to fetch unread emails matching the specified subject criteria, such as “Process document.”

A loop structure is built using the For Each MailMessage activity, which will process all emails fetched from the above step.

---

2. Attachment Extraction and Structured Storage

For each email, the system extracts PDF attachments using the *Save Mail Attachments* activity. The attachments are stored in dynamically generated folders based on the sender’s email address and timestamp. This naming convention ensures uniqueness and prevents overwriting of files.

The folder structure is organized as follows:

Input/Attachments/<sender_email>_<timestamp>/

This structured storage mechanism enhances traceability and supports batch processing of multiple documents.

---

3. Document Preprocessing using OCR

Once the attachment has been extracted, the system goes through each of the PDF files independently, using a file loop (For Each File in Folder) structure.

OCR is done through offline OCR tools like Microsoft OCR or Tesseract.

The purpose of the OCR tool is to transform unstructured content in documents to readable text. This text is stored in a string variable and later on exported to a text file.

The files produced are saved in the following folder:

Output/RawText/<filename>.txt

---

4. Logging and Error Handling Mechanism

To achieve stability and resilience within the system, a Try-Catch pattern is employed for every instance when an email is being processed. This prevents an error from occurring while processing any email from affecting the entire workflow.

The following logging actions help track critical activities in the system, namely:

- Status of email processing
- Total number of attachment processed
- Status of OCR process
- Error messages and exceptions encountered

---

5. Workflow Lifecycle Management

When all the processing is completed, the email will be moved from the Inbox to a specific Processed Folder by using the Move Outlook Mail Message task.

Also, there may be some other methods that could be applied to clean up the already processed files.

---

6. System Architecture Summary

The system is built using a modular design that includes the following modules:

• Email ingestion module

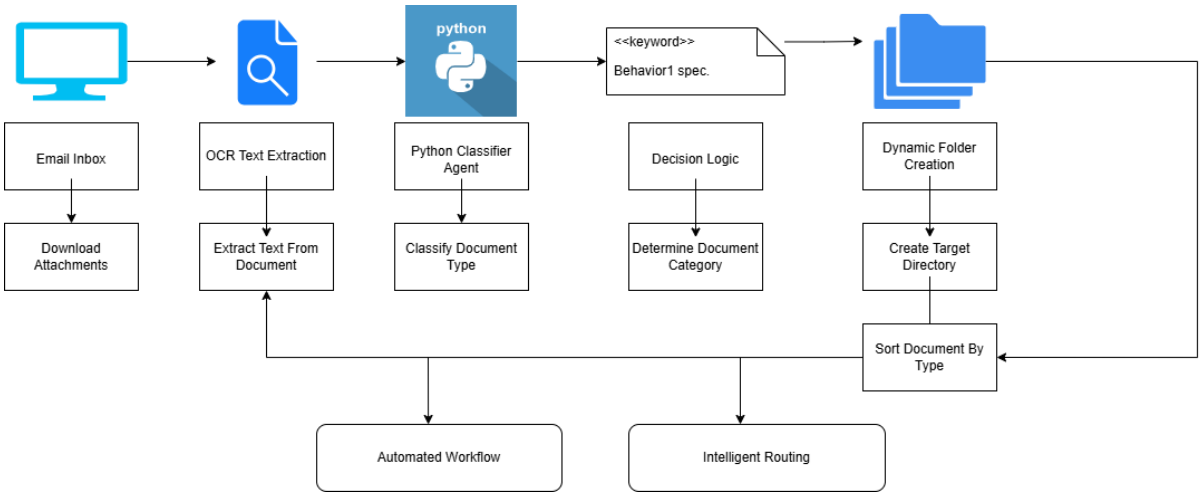
• Attachment processing module

• OCR-based preprocessing module

• Logging & Monitoring module

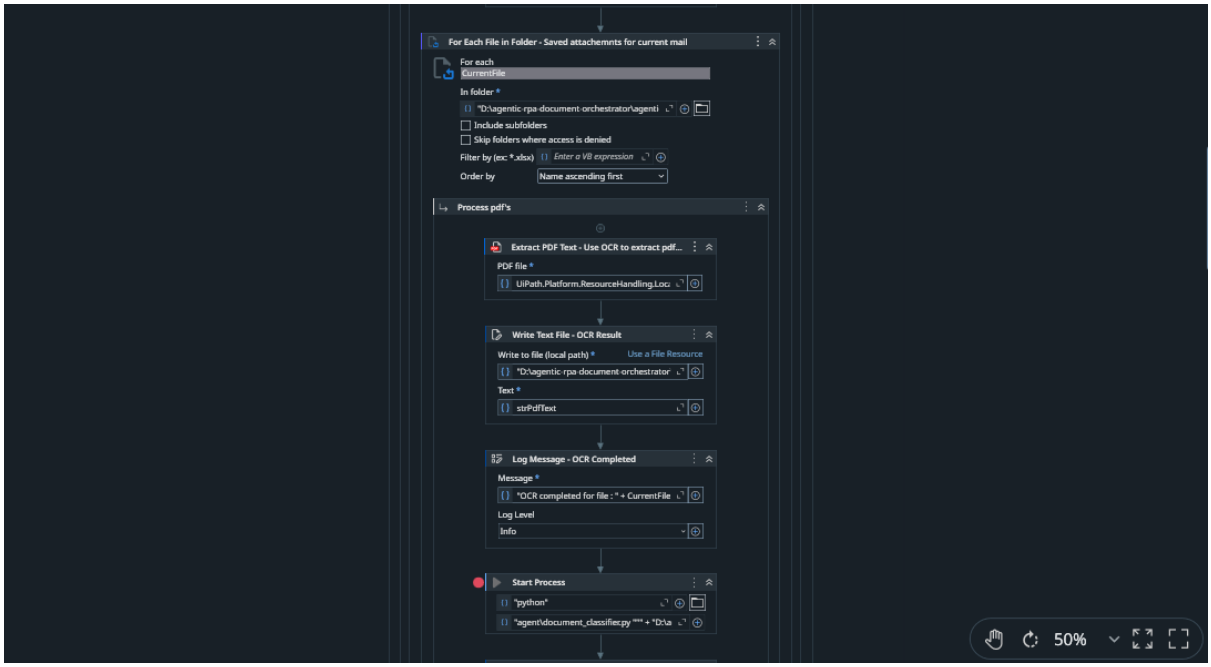
All these modules work independently but together form part of the document processing pipeline.

Flowchart

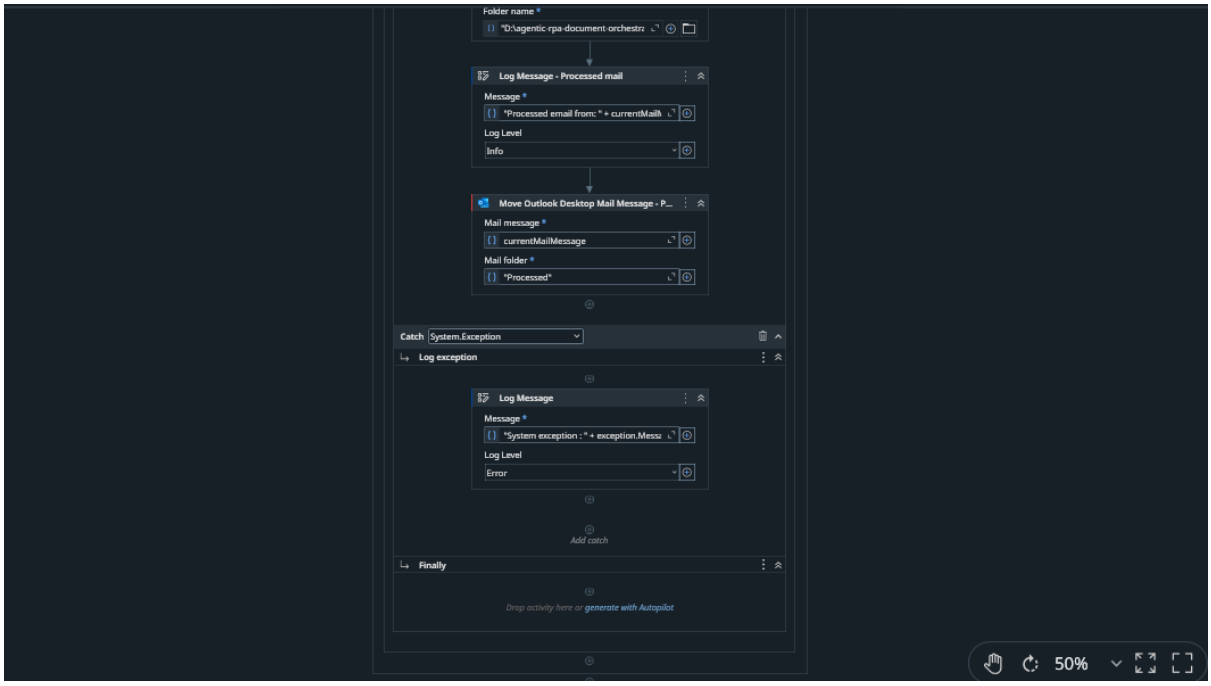


Code Screenshots

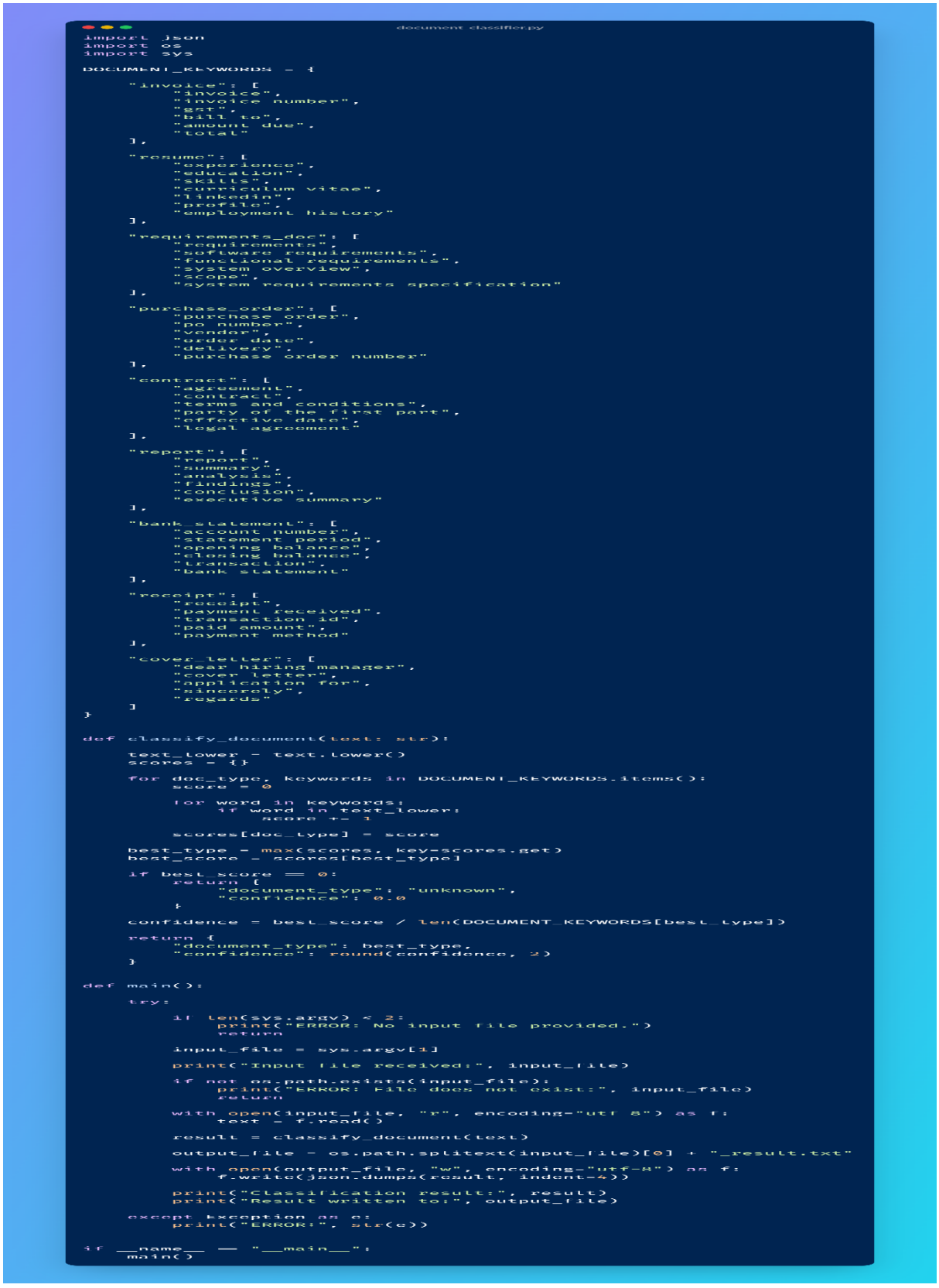
UIPath workflow




Log Message
Message *
"Agent analyzing document..."
Log Level
Info
Delay
Duration *
00h 00m 05.000s
Read Text File
File (local path) *
Use a File Resource
"D:\agentic-rpa-document-orchestrator
Deserialize JSON
JSON string *
strJsonResult
JSON Sample
Log Message
Message *
"Agent detected document type : " + jobj
Log Level
Info
Folder Exists
Folder path *
"D:\agentic-rpa-document-orchestra
Save result
boolFolderExists
50%
boolFolderExists
If
Condition *
boolFolderExists
Then
Move File
From *
CurrentFile.FullName
To
"D:\agentic-rpa-document-orchestra
Overwrite
Else
Create Folder
Folder name *
"D:\agentic-rpa-document-orchestrator
Move File
From *
CurrentFile.FullName
To
"D:\agentic-rpa-document-orchestra
Overwrite
Log Message
Message *
"File moved to : output/" + jobjResult("c
Log Level
50%


Folder name *
"D:\agentic-rpa-document-orchestra
Log Message - Processed mail
Message *
"Processed email from: " + currentMailh
Log Level
Info
Move Outlook Desktop Mail Message - P...
Mail message *
currentMailMessage
Mail folder *
"Processed"
Catch System.Exception
Log exception
Log Message
Message *
"System exception : " + exception.Messa
Log Level
Error
Add catch
Finally
Drop activity here or generate with Autopilot
50%

Python Classifier



```python
import json
import os
import sys

DOCUMENT_KEYWORDS = {

    "invoice": [
        "invoice",
        "invoice number",
        "gst",
        "bill to",
        "amount due",
        "total"
    ],

    "resume": [
        "experience",
        "education",
        "skills",
        "curriculum vitae",
        "linkedin",
        "profile",
        "employment history"
    ],

    "requirements_doc": [
        "requirements",
        "software requirements",
        "functional requirements",
        "system overview",
        "scope",
        "system requirements specification"
    ],

    "purchase_order": [
        "purchase order",
        "po number",
        "vendor",
        "order date",
        "delivery",
        "purchase order number"
    ],

    "contract": [
        "agreement",
        "contract",
        "terms and conditions",
        "party of the first part",
        "effective date",
        "legal agreement"
    ],

    "report": [
        "report",
        "summary",
        "analysis",
        "findings",
        "conclusion",
        "executive summary"
    ],

    "bank_statement": [
        "account number",
        "statement period",
        "opening balance",
        "closing balance",
        "transaction",
        "bank statement"
    ],

    "receipt": [
        "receipt",
        "payment received",
        "transaction id",
        "paid amount",
        "payment method"
    ],

    "cover_letter": [
        "dear hiring manager",
        "cover letter",
        "application for",
        "sincerely",
        "regards"
    ]
}

def classify_document(text: str):

    text_lower = text.lower()
    scores = {}

    for doc_type, keywords in DOCUMENT_KEYWORDS.items():
        score = 0

        for word in keywords:
            if word in text_lower:
                score += 1

        scores[doc_type] = score

    best_type = max(scores, key=scores.get)
    best_score = scores[best_type]

    if best_score == 0:
        return {
            "document_type": "unknown",
            "confidence": 0.0
        }

    confidence = best_score / len(DOCUMENT_KEYWORDS[best_type])

    return {
        "document_type": best_type,
        "confidence": round(confidence, 2)
    }

def main():

    try:

        if len(sys.argv) < 2:
            print("ERROR: No input file provided.")
            return

        input_file = sys.argv[1]

        print("Input file received:", input_file)

        if not os.path.exists(input_file):
            print("ERROR: File does not exist:", input_file)
            return

        with open(input_file, "r", encoding="utf-8") as f:
            text = f.read()

        result = classify_document(text)

        output_file = os.path.splitext(input_file)[0] + "_result.txt"

        with open(output_file, "w", encoding="utf-8") as f:
            f.write(json.dumps(result, indent=4))

        print("Classification result:", result)
        print("Result written to:", output_file)

    except Exception as e:
        print("ERROR:", str(e))

if __name__ == "__main__":
    main()
```

Result

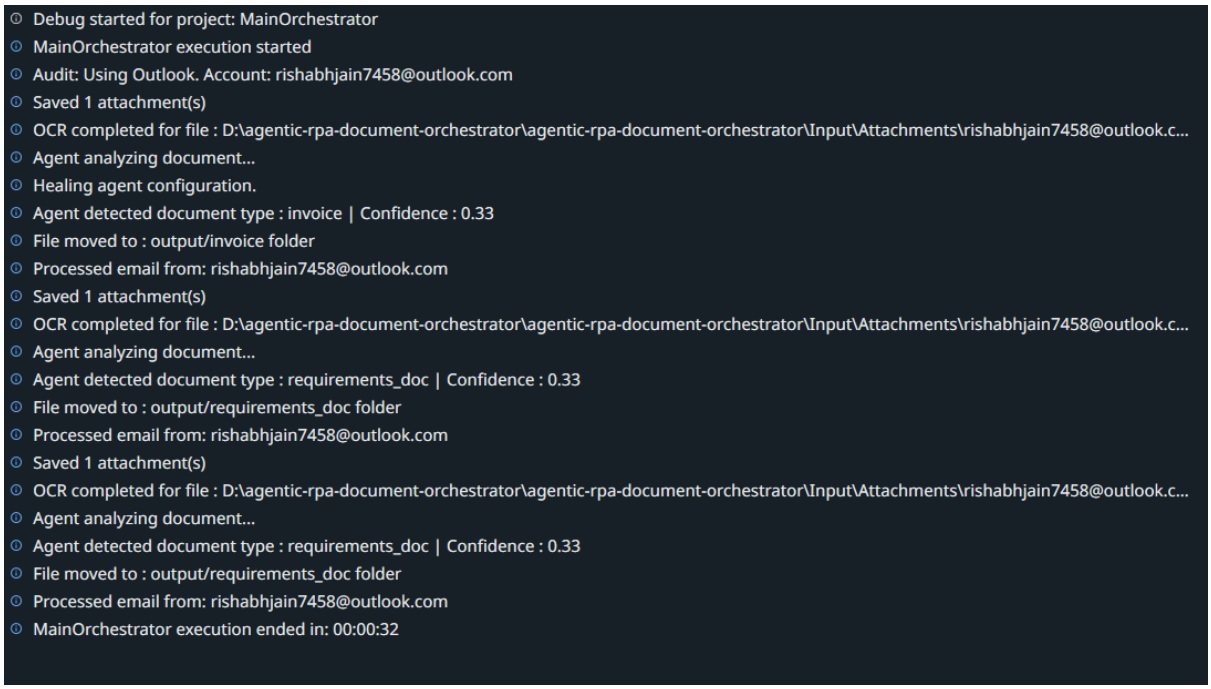


Fig. Final logs

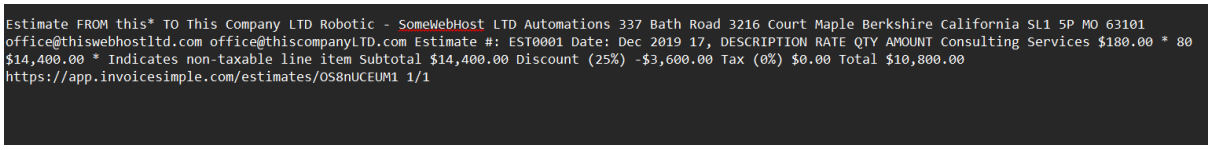


Fig. OCR Result